DAY 32 | POST-READ

LLMOps: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
MLOps vs LLMOps	LLMOps adds prompts, provider models, token costs, judges and drift from provider updates.
When to fine-tune	For consistent behaviour, format or efficiency at volume; not for fresh facts.
Fine-tuning data	Chat JSONL; clean, consistent, PII-free; train and validation split; compare with the base model on the golden set.
Versioning	Prompt, model version, retrieval settings, data versions together in a release manifest.
Cost per request	tokens in x input price + tokens out x output price; prices from configuration.
Latency	Track p50 and p95; output length and model size dominate.
Levers	Trim context, concise outputs, caching, routing to smaller models, capping agent steps.

Part B: Going beyond today

1. Parameter-efficient fine-tuning

For open models, methods such as LoRA train small adapter layers instead of the whole model, making fine-tuning feasible on a single GPU. Several adapters can share one base model.

2. Distillation

Use a large model to produce high-quality outputs, review them, then fine-tune a smaller model on those examples. The small model often handles the narrow task well at a fraction of the cost.

3. Semantic caching

Exact caches only hit identical questions. Semantic caches match similar questions using embeddings. They save more but risk returning a wrong cached answer, so use strict similarity thresholds and exclude personalised answers.

4. Prompt caching by providers

Some providers discount repeated prompt prefixes. Keep stable content (system rules, tool definitions) at the start of prompts so it can be reused.

5. Budgets and alerts

Set daily cost budgets per feature and alert at 80 percent. A runaway agent loop can burn a month's budget in hours.

Part C: Check yourself

- Give one case where fine-tuning is right and one where RAG is right.
- What goes into a release manifest?
- How do you calculate the cost of one request?
- Why can routing reduce cost without hurting quality much?
- What is a risk of semantic caching?

Part D: Practice ideas for tonight

- Create a release manifest for your capstone idea.
- Add a cost and latency summary to the log analysis from Day 31.
- Prepare 50 high-quality examples for a narrow task you could fine-tune.

Coming up on Day 33: deployment architectures, environments, secrets, scalability and inference pipelines.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today